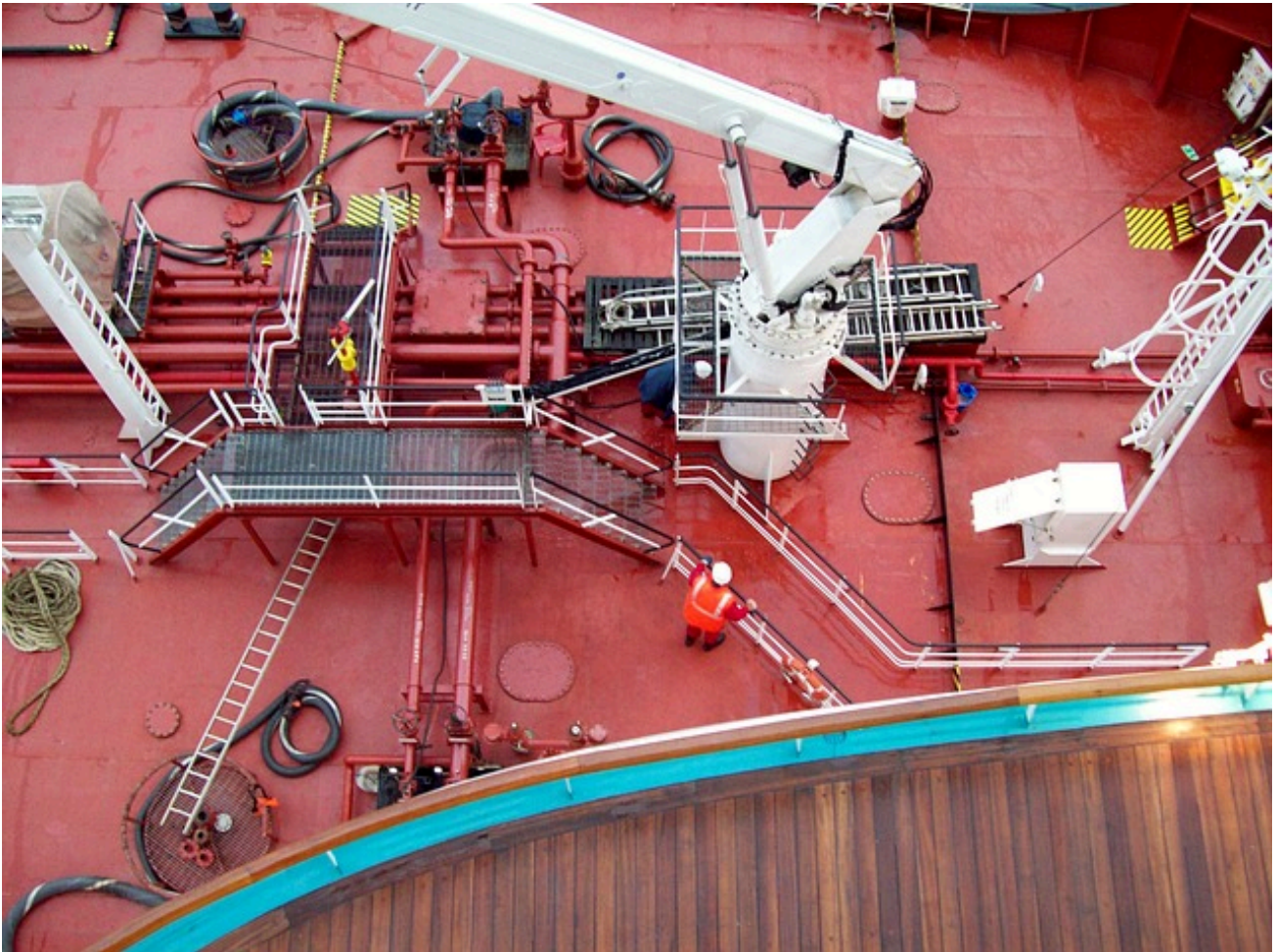


CLOUD NATIVE ECOSYSTEM / КОНТЕЙНЕРЫ / KUBERNETES / МИКРОСЕРВИСЫ / СЕТЬ

Руководство хакера по сетевым технологиям Kubernetes

Вот как работает Kubernetes и Container Networking Interface, а также несколько хакерских приемов, которые помогут изучить его внутреннюю структуру и управлять ею.

27 февраля 2017 года, 10:33 [Ярон Хавив](#)



Изображение для заставки [через](#) Pixabay.

Платформа, которую мы создали в Iguzio, является облачной и использует микросервисы на основе Docker, etcd и собственное управление кластером. Сейчас, когда Kubernetes стал более зрелым, мы постепенно переходим на **систему оркестрации контейнеров Kubernetes**, используя ее расширенные функции, чтобы сосредоточиться на предоставлении уникальных сервисов.

В отличие от большинства облачных приложений, наше работает в режиме реального времени. Мы добиваемся высочайшей производительности за счет низкоуровневого прямого доступа к сети, хранилищу, процессору и ресурсам памяти. Эта задача, связанная с контейнерами и Kubernetes, не так проста и требует по-хорошему старомодного подхода.

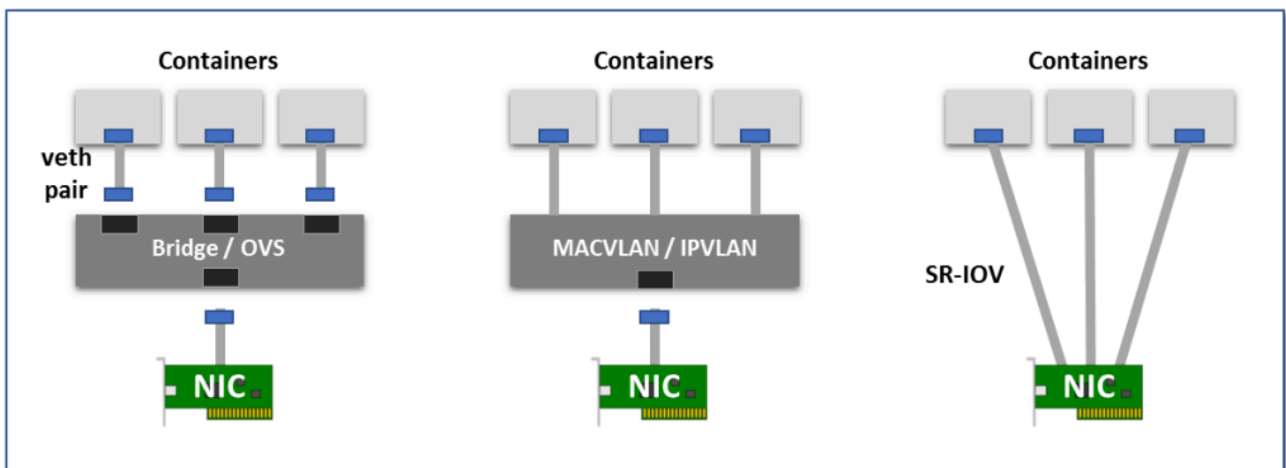
Этот пост — первый в серии. Я расскажу, как работают Kubernetes и **сетевой интерфейс контейнеров**, а также поделюсь некоторыми хакерскими приемами, которые помогут разобраться в их внутреннем устройстве и научиться им управлять. В следующих постах мы расскажем о высокопроизводительном хранилище и приемах межпроцессного взаимодействия (IPC), которые мы используем в контейнерах.

Контейнерные процессы сопоставляются с сетевыми пространствами имен, пространствами имен хранилища и другими пространствами имен. Каждое пространство имен «видит» только часть ресурсов ОС, что гарантирует изоляцию между контейнерами.

На сетевом уровне пространство имён имеет собственный сетевой стек с интерфейсами, таблицами маршрутизации, сокетами и правилами IPTABLE. Интерфейс может принадлежать только одному сетевому пространству имён. Для использования нескольких контейнеров требуется несколько интерфейсов. Другой вариант — создать псевдоинтерфейсы и программно подключить их к реальному интерфейсу (мы также можем сопоставить контейнеры с сетевым пространством имён хоста, как это делается для **демонов**).

Вот несколько вариантов создания и подключения псевдоинтерфейса:

- **Виртуальный мост:** создайте пары виртуальных интерфейсов (veth), один из которых будет находиться в контейнере, а другой — в корневом пространстве имен, и используйте Linux Bridge или **OpenvSwitch** (OVS) для подключения между контейнерами и внешними (реальными) интерфейсами. Использование мостов может привести к дополнительным затратам по сравнению с прямым подключением.
- **Мультиплексирование:** мультиплексирование может осуществляться с помощью промежуточного сетевого устройства, предоставляющего несколько виртуальных интерфейсов, с правилами переадресации пакетов, которые определяют, к какому интерфейсу направляется каждый пакет. MACVLAN назначает MAC-адрес для каждого виртуального интерфейса (исходящие пакеты помечаются этим MAC-адресом, а входящие мультиплексируются в зависимости от MAC-адреса назначения). IPVLAN делает то же самое, но на основе IP-адресов и с использованием одного MAC-адреса, что делает его более удобным для виртуальных машин.
- **Аппаратная коммутация:** большинство современных сетевых адаптеров поддерживают виртуализацию ввода-вывода с одним корневым устройством (**SR-IOV**), что позволяет создавать несколько виртуальных устройств. Каждое виртуальное устройство представляется как отдельное устройство PCI. Оно может иметь собственную виртуальную локальную сеть и аппаратно реализованную ассоциацию QoS. Технология SR-IOV обеспечивает производительность на уровне «голого железа», но обычно недоступна в общедоступном облаке.



Режимы виртуальной сети: бриджинг, мультиплексирование и SR-IOV.

🔍 Масштабирование

Во многих случаях пользователям требуется создать логические сетевые подсети, охватывающие несколько сегментов сети уровня 2/3. Для этого требуются протоколы оверлейной инкапсуляции (наиболее распространенный из них — **VXLAN**, который инкапсулирует оверлейный трафик в пакеты UDP). VXLAN может приводить к увеличению накладных расходов, а несколько сетей VXLAN от разных производителей обычно несовместимы из-за отсутствия стандартизации в плоскости управления.

1. [Cursor незаметно приобретает Continue](#) — альтернативу GitHub Copilot с открытым исходным кодом
2. [OpenClaw и Hermes сходятся во мнении о том, что такое агент. Но расходятся в вопросе о том, что его контролирует.](#)
3. [Публичный ключ Sentry](#) — это все, что нужно для взлома Claude Code, Cursor и Codex
4. [Gemini CLI против. Antigravity: что работает, а не что указано в спецификации](#)
5. [Разработчики теперь проверяют код, который не писали и который могут не понимать](#)

Kubernetes также активно использует IPTABLES и NAT для перехвата трафика, поступающего на логический/виртуальный адрес, и перенаправления его на соответствующий физический адрес назначения. **Сетевые решения для контейнеров**, такие как Flannel, **Calico** и Weave, используют veth с мостом/маршрутизатором и некоторыми оверлейными функциями или функциями маршрутизации/NAT.

Ознакомьтесь с этим подробным практическим руководством и инструкцией по использованию различных сетевых возможностей Linux.

Помимо ожидаемых накладных расходов на обработку пакетов, виртуальные сети влекут за собой скрытые издержки и могут негативно влиять на параллелизм процессора и памяти, например:

- Сетевые адаптеры направляют трафик на ядра на основе заголовков. Если заголовок изменен, трафик будет направляться не на те ядра, что может снизить эффективность использования памяти и процессора.
- Сетевые адаптеры формируют пакеты и передают контрольные суммы на аппаратное обеспечение, что позволяет сэкономить много циклов работы процессора и памяти. Если оверлейные пакеты формируются программно или если у нас есть оверлей (контейнер) поверх оверлея (облако/IaaS), это приведет к снижению производительности. Новые сетевые адаптеры могут формировать пакеты VXLAN на аппаратном уровне, но оверлейное решение должно использовать их напрямую.

Некоторые приложения (например, Iguazio) используют расширенные возможности сетевых адаптеров, такие как RDMA, библиотеку **DPDK** для быстрой обработки сетевых данных или шифрование, чтобы разгрузить обмен сообщениями, лучше контролировать параллелизм процессора, уменьшить количество прерываний или исключить копирование данных в память. Это возможно только при использовании прямого сетевого интерфейса или виртуального интерфейса SR-IOV.

Игра с сетевыми пространствами имён POD

Не существует простого способа увидеть сетевые пространства имён, поскольку Kubernetes и Docker их не регистрируют («ip netns» не работает с Kubernetes и Docker). Но мы можем использовать несколько хитростей, чтобы видеть, отлаживать, управлять и настраивать сетевые пространства имён POD с хоста.

Сетевые пространства имен перечислены в `/proc/<PID>/ns/net`, поэтому нам нужно найти идентификатор процесса (PID) для нашего POD. Сначала найдем идентификатор контейнера с помощью приведенной ниже команды и возьмем только первые 12 цифр:

```
kubectl get po <POD-NAME> -o jsonpath='{.status.containerStatuses[0].containerID}' | cut -c 10-
```

Теперь давайте воспользуемся командами Docker, чтобы найти PID:

```
:ID=$(docker inspect --format '{{.State.Pid}}' <ContainerID>)
```

```
nsenter -t ${PID} -n ip addr
```

...покажет нам все интерфейсы POD вместе с их IP-адресами. Затем мы можем использовать другие команды, например `ping` или `curl`, для проверки подключения, а также привилегированные операции или те, которые не установлены в контейнере POD, для мониторинга, отладки или настройки этого POD (например, `ip route`, `nslookup` и т. д.).

```
$ nsenter -t 13537 -n ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if45: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
   link/ether 0a:58:0a:f4:00:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
   inet 10.244.0.2/24 scope global eth0
       valid_lft forever preferred_lft forever
   inet6 fe80::30d0:a8ff:fe43:fe80/64 scope link tentative dadfailed
       valid_lft forever preferred_lft forever
```

 Zoom

Если нас не устраивает один интерфейс на POD, мы можем просто взять или создать интерфейсы в пространстве имен хоста и назначить их POD:

```
ip link set netns ${PID} <IFNAME>
```

А если мы захотим их вернуть, то просто отправим их обратно на хостинг:

```
nsenter -t ${PID} -n ip link set <IFNAME> netns 1
```

Автоматизация сетевых подключений в Kubernetes с помощью CNI

Kubernetes использует подключаемые модули CNI для **оркестровки сетевых подключений**. Каждый раз при инициализации или удалении POD вызывается подключаемый модуль CNI по умолчанию с конфигурацией по умолчанию. Этот подключаемый модуль CNI создает псевдоинтерфейс, подключает его к соответствующей базовой сети, задает IP-адрес, маршруты и сопоставляет его с пространством имен POD.

К сожалению, Kubernetes по-прежнему поддерживает только один интерфейс CNI на POD с одной конфигурацией для всего кластера. Это очень ограничивает возможности, поскольку мы можем захотеть настроить несколько сетевых интерфейсов на POD, потенциально используя разные оверлейные решения с разными политиками (подсеть, безопасность, QoS).

Let's see how we can bypass those limitations.

When the Kubelet Kubernetes local agent configures POD networking, it looks for a CNI json configuration file in `/etc/cni/net.d/` and a relevant plug-in binary (based on the type attribute) located in `/opt/cni/bin/`. A CNI plug-in can invoke a secondary IP Address Management (IPAM) plug-in to set the IP address per interface (e.g. `host-local` or `dhcp`). It is also possible to use other paths through Kubelet command options:

```
{
  "name": "mynet",
  "type": "bridge",
  "bridge": "cni0",
  "isGateway": true,
  "ipMasq": true,
  "ipam": {
    "type": "host-local",
    "subnet": "10.22.0.0/16",
    "routes": [
      { "dst": "0.0.0.0/0" }
    ]
  }
}
```

[Zoom](#)

Kubelet invokes the CNI plug-in with environment variables containing command parameters (CNI_ARGS, CNI_COMMAND, CNI_IFNAME, CNI_NETNS, CNI_CONTAINERID, CNI_PATH) and streams the json.conf file through stdin. The plug-in responds with json output text, describing the results and status. See more [detailed explanation and examples here](#). It's relatively simple to develop your own CNI plug-in if you know the Go programming language, as the framework does much of the magic and you can use or extend one of the existing plug-ins here.

Kubelet will pass the POD name and namespace as part of the CNI_ARGS variable (for example "K8S_POD_NAMESPACE=default;K8S_POD_NAME=mytests-1227152546-vq7kw;"). We can use this to customize the network configuration per POD or POD namespace (e.g. put every namespace in a different subnet). Future Kubernetes versions will treat networks as equal citizens and include network configuration as part of the POD or namespace spec just like memory, CPUs and volumes. For the time being, we can use annotations to store configuration or record POD networking data/state.

The CNI plug-in does its own logic once invoked. It can also attach multiple network interfaces to the same POD and bypass the current limitations, with one caveat: Kubernetes will only be aware of the one we report in the results (which is used for service discovery and routing). I recently stumbled upon a great Intel open source CNI plug-in called [Multus](#) which does exactly that. Linux programmer [Doug Smith](#) wrote a detailed [Multus walk-through](#) after a recent Slack chat.

Multus accepts a hierarchical conf file with an array of CNI definitions. It will configure each POD with multiple interfaces, one per definition, and we can specify which interface is the "masterplugin" to be recognized by Kubernetes. We use this as a baseline coupled with the POD name and POD annotations to create flexible and unique network configurations per POD.

The same Intel git also includes another interesting CNI driver for [SR-IOV](#) and [DPDK](#) support.

Summary

I love cloud-native and microservices, the impact they are having on agile and continuous application delivery is immense. However, it seems like container networking projects are still nascent and just taking their first steps, which is essentially no more than solving the challenge of cross-segment/cloud connectivity. These projects still require customization to make them fit the broader application base and higher-performance. Hopefully, it will quickly improve and get to the functionality level of other more mature [software-defined networking solutions](#) like OpenStack Neutron or VMware NSX.

How do we make all these different overlay and underlay control planes interoperable, allowing two different vendor/cloud solutions in both ends of a logical network? This is the \$64,000 question and I welcome your feedback. It's clearly a key area that requires more attention and standardization.

TNS



Yaron Haviv, the CTO and founder of Iguazio, is a serial entrepreneur who has deep technological experience in the fields of big data, cloud, storage and networking. Prior to Iguazio, Yaron was the Vice President of Datacenter Solutions at Mellanox,...

[Read more from Yaron Haviv →](#)

TNS owner Insight Partners is an investor in: Docker, Root.

SHARE THIS STORY



TRENDING STORIES

1. Cursor quietly acquires Continue, an open-source alternative to GitHub Copilot
2. OpenClaw and Hermes agree on what an agent is. They disagree on what controls it.
3. A public Sentry key is all it takes to hijack Claude Code, Cursor, and Codex
4. Gemini CLI vs. Antigravity: What works, not the spec sheet
5. Developers are now validating code they didn't write — and may not understand

INSIGHTS FROM OUR SPONSORS

Everything we announced during AI Readiness Innovation Week

25 June 2026

Fewer CVEs, more accurate findings: Wiz now scans Chainguard Libraries for Python and Java

25 June 2026

Chainguard Repository adds new policies, Chainguard Libraries for JavaScript is GA

25 June 2026



NUMA Explained: Why Memory Distance Slows Your VMs

23 June 2026

Docker-in-Docker Security Risks and How to Escape Them

17 June 2026

Kubernetes User Namespaces Don't Fix the Shared Kernel

4 June 2026



AI Test Automation: Fix the 3 New Testing Bottlenecks

17 June 2026

Test Observability: Fix Flaky Kubernetes Tests (2026)

11 June 2026

Testkube AI Launch: New Use Cases, OAuth, and Open Source Analysis

9 June 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the most recent
TNS articles in your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds

 roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

© The New Stack 2026

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)